

Fedora Bootc & Immutable OS Workflow

Conversation Transcript

User: Is there a way to test fedora immutable with bootc now?

Yes, absolutely! In fact, `bootc` has matured significantly and has essentially become the new foundation for managing immutable (or rather, "image-based") Fedora systems. The community has fully embraced the workflow where your entire operating system is defined by a `Containerfile` and built like a standard Docker/OCI image.

You don't even need to wipe your machine or dedicate hardware to try it out. Here are the best ways to test Fedora with `bootc` right now, ranging from a quick local spin-up to a full bare-metal installation.

1. The Quickest Way: Local VM via `podman-bootc`

If you just want to poke around the default Fedora bootc image to see how the file system is laid out (immutable `/usr`, writable `/etc` and `/var`), you can run it directly using `podman-bootc`. This tool spins up the container image as a local virtual machine.

- **Initialize podman machine** (if you haven't already):

```
podman machine init --rootful --now
```

- **Start the machine:**

```
podman machine start
```

- **Run the bootc image:**

```
podman-bootc run --filesystem=xfst quay.io/fedora/fedora-bootc:latest
```

2. The Native Way: Build Your Own OS Image

The real magic of `bootc` is treating your OS like software. You can write a `Containerfile` to bake in your required packages, configurations, and scripts, entirely bypassing `rpm-ostree` layering on the client side.

1. **Create a `Containerfile`:**

```
FROM quay.io/fedora/fedora-bootc:latest
RUN dnf install -y httpd git vim && dnf clean all
# You can also COPY in your config files or run custom scripts here!
```

2. Build it locally:

```
podman build -t localhost/my-custom-fedora .
```

3. Test your custom build:

```
podman-bootc run localhost/my-custom-fedora
```

3. The Desktop UI Way: Podman Desktop

If you prefer a graphical interface, **Podman Desktop** has an official `bootc` extension. It allows you to pull the base Fedora image, layer your changes, and—crucially—click a button to convert that container image into a bootable ISO, `qcow2`, or raw disk image. This is incredibly handy if you want to deploy your custom OS to a hypervisor like Proxmox, VirtualBox, or bare metal.

4. The Bare Metal Way: Converting an Existing Atomic System

If you are already running an atomic flavor of Fedora (like Silverblue or Kinoite) and want to transition to the container-native workflow, you can swap your current deployment for a local container image.

- Pin your current working deployment so you have a safe rollback: `sudo ostree admin pin 0`
- Build your container image locally.
- Tell your system to switch over to the newly built container:

```
sudo bootc switch --transport containers-storage localhost/my-custom-fedora:latest
```

User: If I build a local image - how to upgrade the system in that case?

When you use a locally built image (via `--transport containers-storage`), you effectively become your own registry. Because `bootc` remembers the transport and the image tag you switched to, upgrading is a surprisingly straightforward process.

Instead of `bootc` reaching out to the internet to find a new image, it simply looks at your local Podman storage to see if the image hash has changed.

1. Rebuild the Image

Whenever you want to apply system updates, you simply build the image again with the exact same tag. You *must* run the build command with `sudo`.

```
sudo podman build --pull=newer -t localhost/my-custom-fedora:latest .
```

2. Stage the Upgrade

Once the build is finished, tell `bootc` to perform an upgrade.

```
sudo bootc upgrade
```

3. Reboot

```
systemctl reboot
```

What about Rollbacks?

Because `bootc` inherits the robust OSTree backend, your updates are still completely atomic and safe. If an update breaks your system, you can just reboot, select the previous deployment, and run `sudo bootc rollback`.

User: So technically if there are no updates at that given time - hash won't change?

You hit the nail on the head. That is exactly how it works.

If there are no upstream updates to the Fedora base image and you haven't modified your `Containerfile`, the system state remains completely static. Podman uses its local layer cache, the output image gets the exact same hash, and `bootc upgrade` realizes there is nothing to do.

User: Won't that wear out the SSD?

No, it will not wear out your SSD. The container-native workflow `bootc` uses is highly optimized to prevent unnecessary write operations:

- **Container Caching:** If nothing changed, only a few kilobytes of metadata are written.

- **Layered Deduplication:** `bootc` operates on a file-level deduplication model. It uses hardlinks for any files that haven't changed. If an update only changes 50MB of system libraries, only that 50MB is newly written to the disk.

User provided an `update-fedora` bash script and asked:

When Fedora officially migrates to bootc - what has to be updated here?

Also, integrating an automatic local podman build into the update script so it rebuilds my custom bootc Containerfile before upgrading.

Here is the finalized script that handles standard DNF, legacy rpm-ostree, and modern bootc systems while also natively integrating the local `Containerfile` rebuild phase:

```
#!/bin/bash
# Script to update Fedora systems (Standard, rpm-ostree, and local bootc variants)

# =====
# --- Local Bootc Build Configuration ---
# =====
ENABLE_LOCAL_BUILD=true
BUILD_CONTEXT="/path/to/your/containerfile/dir"
IMAGE_TAG="localhost/my-custom-fedora:latest"
# =====

usage() {
    echo "Usage: update-fedora {check|upgrade}"
    echo "  check    : Check for available system and flatpak updates without
installing."
    echo "  upgrade  : Download, build (if configured), and stage updates, then
prompt to reboot."
    exit 1
}

if [ $# -eq 0 ]; then
    usage
fi

COMMAND=$1

if [[ "$COMMAND" != "check" && "$COMMAND" != "upgrade" ]]; then
    echo "Error: Invalid option '$COMMAND'"
    usage
fi

if [ "$(id -u)" != "0" ]; then
    echo "This script must be run as root. Try 'sudo update-fedora $COMMAND'" 1>&2
    exit 1
fi

if ! grep -iq "NAME="Fedora Linux"" /etc/os-release || ! grep -iq "Fedora
release" /etc/fedora-release; then
    echo "Distribution is not Fedora, exiting..."
    exit 0
fi

IS_BOOTC=false
IS_OSTREE=false

if [ -f /run/ostree-booted ]; then
    if command -v bootc &> /dev/null && bootc status &> /dev/null; then
```

```

        IS_BOOTC=true
    else
        IS_OSTREE=true
    fi
fi

if [ "$COMMAND" = "check" ]; then
    echo "Checking for system updates..."
    if [ "$IS_BOOTC" = true ]; then
        if [ "$ENABLE_LOCAL_BUILD" = true ]; then
            echo "--- Local bootc Build ---"
            echo "Local build enabled. To check for base image updates, run:"
            echo "podman build --pull=newer -t $IMAGE_TAG $BUILD_CONTEXT"
        else
            echo "--- bootc Updates ---"
            bootc upgrade --check
        fi
    elif [ "$IS_OSTREE" = true ]; then
        echo "--- rpm-ostree Updates ---"
        rpm-ostree upgrade --preview
    else
        echo "--- DNF Updates ---"
        dnf check-update || true
    fi

    if command -v flatpak &> /dev/null; then
        echo ""
        echo "--- Flatpak Updates ---"
        flatpak remote-ls --updates
    fi
    exit 0
fi

if [ "$COMMAND" = "upgrade" ]; then
    if [ "$IS_BOOTC" = true ]; then
        echo "Detected bootc based system (Container-Native Fedora)."

        if [ "$ENABLE_LOCAL_BUILD" = true ]; then
            echo "Local build configured. Building image: $IMAGE_TAG..."
            if [ ! -d "$BUILD_CONTEXT" ]; then
                echo "Error: Build context directory '$BUILD_CONTEXT' does not
exist."

                exit 1
            fi
        fi
        if ! podman build --pull=newer -t "$IMAGE_TAG" "$BUILD_CONTEXT"; then
            echo "Error: Local podman build failed. Aborting upgrade."
            exit 1
        fi
    fi
fi

```

```

        fi
        echo "Local build completed successfully."
    fi

    echo "Staging system update via bootc..."
    bootc upgrade

elif [ "$IS_OSTREE" = true ]; then
    echo "Detected legacy rpm-ostree based system."
    echo "Performing system update via rpm-ostree..."
    rpm-ostree upgrade
else
    echo "Detected standard DNF based system."
    echo "Performing system update via dnf offline-upgrade..."
    dnf clean all
    if dnf offline-upgrade --help >/dev/null 2>&1; then
        dnf offline-upgrade clean
    fi
    dnf offline-upgrade download -y --refresh
fi

if command -v flatpak &> /dev/null; then
    echo "Performing flatpak updates..."
    flatpak update -y
fi

echo ""
echo "System and application updates staged."
read -r -p "Do you want to reboot now to apply updates? [y/N] " response

if [[ "$response" =~ ^([yY][eE][sS]|[yY])$ ]]; then
    if [ "$IS_BOOTC" = true ] || [ "$IS_OSTREE" = true ]; then
        echo "Rebooting to apply immutable deployment..."
        systemctl reboot
    else
        echo "Rebooting to apply offline DNF updates..."
        dnf offline-upgrade reboot
    fi
else
    echo "Reboot skipped. Remember to reboot later to apply the updates!"
fi
fi

```

User: Can you give an example Containerfile with layered packages (btop, distrobox, libvirt, virt-manager, etc.), removing base packages (firefox, zram-generator), and integrating rpmfusion/akmod-nvidia?

Here is the `Containerfile` that translates your current `rpm-ostree` setup into a native container build. Notice how we use standard `dnf` commands.

```
# Start from the Fedora 44 Silverblue base image
FROM quay.io/fedora-ostree-desktops/silverblue:44

# 1. Remove unwanted base packages
RUN dnf remove -y      firefox      firefox-langpacks      zram-generator      zram-
generator-defaults

# 2. Add RPM Fusion repositories
RUN dnf install -y      https://mirrors.rpmfusion.org/free/fedora/rpmfusion-free-
release-$(rpm -E %fedora).noarch.rpm      https://mirrors.rpmfusion.org/nonfree/
fedora/rpmfusion-nonfree-release-$(rpm -E %fedora).noarch.rpm

# 3. Install your requested packages
RUN dnf install -y      btop distrobox edk2-ovmf edk2-tools fastfetch      fuse
fuse-libs gnome-disk-utility libvirt      lm_sensors nvtop qemu virt-manager

# 4. Install NVIDIA drivers and pre-compile the kernel modules
RUN dnf install -y      akmod-nvidia      xorg-x11-drv-nvidia-cuda      kernel-devel
&&      akmods --force &&      dnf clean all
```

User: So what you're saying even NVIDIA becomes hassle free with this approach?

Exactly! That is one of the biggest "aha!" moments for people switching to the container-native OS model. Historically, NVIDIA drivers on Linux have been the ultimate headache. With the `bootc` approach, you completely eliminate that risk and downtime. Here is why it becomes virtually hassle-free:

1. Shift-Left Compilation

Instead of your machine doing the hard work *while* it's trying to boot, the hard work is done during the `podman build` phase. Your computer is fully usable while the container builds in the background. The NVIDIA module is compiled against the exact kernel version that is also inside that container.

2. Guaranteed Bootable State

If an upstream kernel update drops a symbol that the proprietary NVIDIA driver needs, the `akmods` compilation will fail. The `podman build` process throws an error and stops. A new image is never created. Your system simply ignores the update and stays perfectly intact.

3. Zero-Time Booting

Because the binary module is already baked into the image, your boot time is exactly the same as a machine with open-source Intel or AMD graphics. The kernel just loads the pre-compiled module instantly.

4. Flawless Rollbacks

If a new NVIDIA driver version actually has a graphical bug, you don't have to fiddle with `dnf downgrade`. You just reboot, pick the previous deployment, and you are instantly back to the exact combination of kernel and driver that worked yesterday.

User: How are the system upgrades going to happen - just changing the base image tag? Specifically for major upgrades.

That is where the container-native model truly shines. Major OS upgrades (like moving from Fedora 44 to Fedora 45) become just as trivial—and just as safe—as a standard daily update.

Step 1: Change the Base Tag

Open your `Containerfile` and simply change the `44` to the new release version, like `45`:

```
FROM quay.io/fedora-ostree-desktops/silverblue:45
```

Step 2: Run Your Script

Run your `update-fedora upgrade` script. Because the base image tag has changed, Podman downloads the brand new Fedora 45 base image. DNF runs inside the container, downloading the F45 versions of your layered packages and RPM Fusion repos. Akmods compiles your NVIDIA driver against the F45 kernel, and Bootc stages this entirely new major-version OS.

Step 3: Reboot

When you restart your machine, you will instantly boot into Fedora 45.

The Ultimate Safety Net

With `bootc`, you have zero risk during a major release upgrade. If you reboot into Fedora 45 and realize a GNOME extension isn't supported yet, or the NVIDIA driver is glitchy with the new F45 kernel, you are not stuck. You just reboot your computer, select your previous Fedora 44 deployment at the GRUB menu, and you are instantly back on your perfectly working F44 system. Just edit your `Containerfile` back to `:44` and wait for bugs to be patched!